Day 10: React.js Introduction

Concepts, hands-on tasks and practice

1. Concepts

What is React? A JavaScript library (by Meta) for building user interfaces out of small, reusable pieces called components. It focuses only on the UI layer.

Advantages of React

- **Component-based:** build once, reuse everywhere
- **Virtual DOM:** React updates only what changed, so the UI stays fast
- **Declarative:** you describe what the UI should look like, and React handles DOM updates
- **Large ecosystem:** routing, state management, UI libraries, strong community
- **One-way data flow:** data moves parent to child, which makes debugging easier

Components: Independent, reusable building blocks (Header, Footer, Button, etc.). An app is a tree of components.

JSX: HTML-like syntax inside JavaScript. Key rules:

- Return a single root element (or a fragment <>...</>)
- Use `className` instead of `class`
- Close all tags (`<img />`)
- Put JavaScript expressions inside `{ }`

Functional components: Plain JavaScript functions that return JSX. Names must start with a capital letter.

Props: Read-only inputs passed from a parent to a child, like function arguments. A child must never modify its own props.

Vite: A fast build tool and dev server (pronounced "veet"). It starts instantly and has hot module replacement, so changes appear in the browser without a reload. It is now the standard way to start a React project, replacing Create React App.

2. Task: Create the Project

Node.js must be installed. Check with `node -v` (use a current LTS version).

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

Open the URL shown in the terminal (usually `http://localhost:5173`). You should see the Vite + React starter page.

3. Project Structure

```
my-react-app/
|-- public/          # Static files served as-is (favicon, images)
|-- src/             # Your application code
|   |-- assets/      # Images, icons imported into components
|   |-- App.jsx      # Root component
```

```
|   |-- App.css
|   |-- main.jsx     # Entry point: mounts React into the page
|   `-- index.css    # Global styles
|-- index.html       # The single HTML page (has <div id="root">)
|-- package.json     # Dependencies and scripts
`-- vite.config.js   # Vite configuration
```

public vs src

- `public/` files are not processed. Reference them by path, e.g. `/logo.png`.
- `src/` files are bundled by Vite. Import them into your code and they get optimized.

Flow: index.html loads main.jsx, which renders <App /> into .

4. Task: Create Header, Footer and Home Components

Create the folder `src/components/` and add three files.

src/components/Header.jsx

```
function Header({ title }) {
  return (
    <header className="header">
      <h1>{title}</h1>
      <nav>
        <a href="#home">Home</a> | <a href="#about">About</a>
      </nav>
    </header>
  );
}

export default Header;
```

src/components/Home.jsx

```
function Home({ name, course }) {
  return (
    <main className="home">
      <h2>Welcome, {name}!</h2>
      <p>You are learning: {course}</p>
    </main>
  );
}

export default Home;
```

src/components/Footer.jsx

```
function Footer({ text }) {
  return (
    <footer className="footer">
      <p>{text}</p>
    </footer>
  );
}

export default Footer;
```

5. Task: Pass Data Using Props

Replace src/App.jsx

```
import Header from "./components/Header";
import Home from "./components/Home";
import Footer from "./components/Footer";
import "./App.css";

function App() {
  return (
    <>
      <Header title="My React App" />
      <Home name="Rahul" course="React.js" />
      <Footer text={`(c) ${new Date().getFullYear()} My React App`} />
    </>
  );
}

export default App;
```

Here **App** (parent) passes `title`, `name`, `course` and `text` down to the child components as props.

Replace src/App.css

```
.header { background: #282c34; color: white; padding: 1rem; }
.header nav a { color: #61dafb; text-decoration: none; margin: 0 0.5rem; }
.home { padding: 2rem; text-align: center; }
.footer { background: #f0f0f0; padding: 1rem; text-align: center; }
```

You can also clear the default styles in `index.css` if the layout looks off.

6. Run the Application

```
npm run dev
```

Expected output: the browser shows a dark header with the title and links, a "Welcome, Rahul!" section in the middle, and a footer at the bottom. The React application runs successfully with basic reusable components.

7. Practice Ideas

1. Make the nav links an array in **App**, pass it to **Header** as a prop, and render it with `.map()`.
2. Create a reusable **Card** component taking `title` and `description` props, and render it three times with different data.
3. Add a default prop value using destructuring: `function Footer({ text = "All rights reserved" })`.
4. Change a prop value in App.jsx and watch the browser update instantly (hot reload).

Common Mistakes

- Lowercase component names (<header /> is an HTML tag, but <Header /> is your component)
- Forgetting `export default` or the `import` line
- Using `class` instead of `className`
- Trying to modify props inside a child component